

Project Report

1. Abstract / Executive Summary

This project focuses on solving a multi-label text emotion classification problem from the “2025 SEP DL GenAI Project” Kaggle competition. Each text sample may contain more than one emotion, making it a multi-label classification task.

To approach this problem, I built and compared three different models:

1. A **custom BiLSTM model** built completely from scratch using PyTorch.
2. A **fine-tuned BERT-base model** using Hugging Face Transformers.
3. A **fine-tuned RoBERTa-base model**, which achieved the best performance.

All models were trained and logged using Weights and Biases (W&B). The RoBERTa model achieved the highest validation micro-F1 score (approximately 0.859). The project also involved tokenization, deep learning experimentation, KaggleHub model uploads, and a clean inference pipeline.

Key learnings include understanding preprocessing, tokenization strategies, sequence modeling, transformer fine-tuning, and model comparison.

2. Introduction

2.1 Problem Statement

The Kaggle competition provides short text samples and asks participants to classify five possible emotion labels: anger, fear, joy, sadness, and surprise. Each sample can express multiple emotions simultaneously, so the task is a multi-label text classification problem. Models must output five binary predictions per sample.

2.2 Project Objectives

The main goals of this project were:

- To study the dataset through Exploratory Data Analysis (EDA).
- To build at least three deep learning models, including a custom model and transformer-based models.
- To fine-tune pretrained models using Hugging Face Transformers.
- To monitor training using Weights & Biases.
- To upload trained models through KaggleHub.
- To create a complete inference pipeline for generating Kaggle submissions.

2.3 Report Structure

The remaining sections discuss the dataset and preprocessing, tokenization strategy, detailed model architectures, experiments, performance comparison, conclusions, and references.

3. Dataset & Preprocessing

3.1 Dataset Description

The dataset consists of three files:

- **train.csv**: 6827 rows containing text and five binary emotion labels.
- **test.csv**: 1707 rows containing only the text column.
- **sample_submission.csv**: template for submission formatting.

Emotion labels are independent and not mutually exclusive.

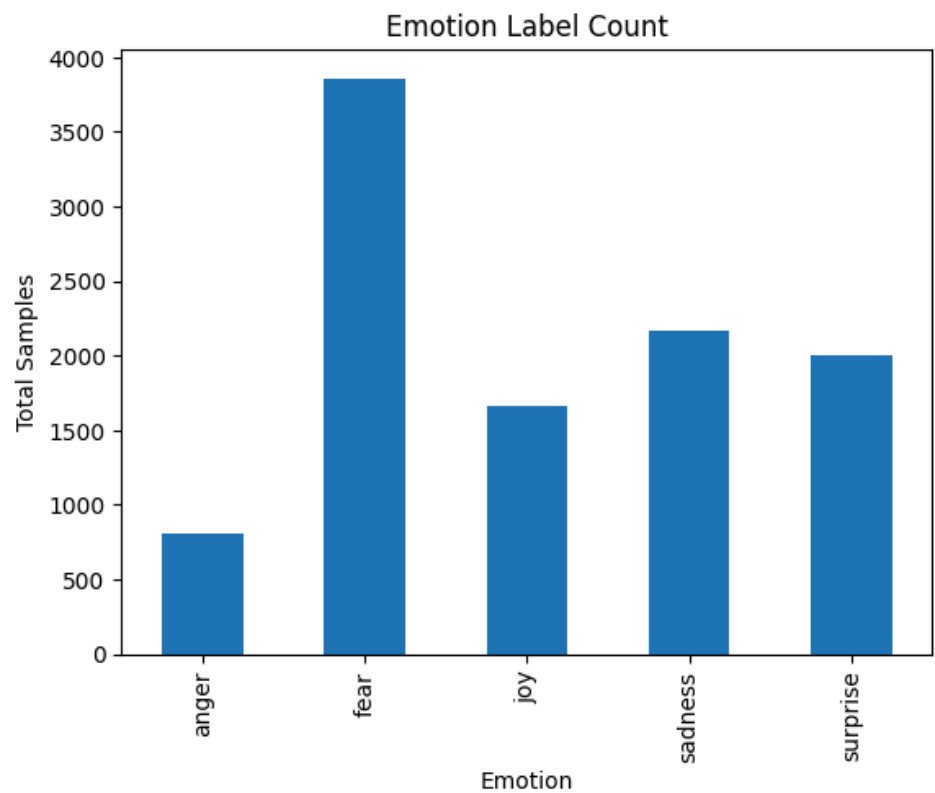
Space for table of sample rows:

	id	text	anger	fear	joy	sadness	surprise	emotions
0	0	the dentist that did the work apparently did a...	1	0	0	1	0	['anger' 'sadness']
1	1	i'm gonna absolutely ~~suck~~ be terrible duri...	0	1	0	1	0	['fear' 'sadness']
2	2	bridge: so leave me drowning calling houston, ...	0	1	0	1	0	['fear' 'sadness']
3	3	after that mess i went to see my now ex-girlfr...	1	1	0	1	0	['anger' 'fear' 'sadness']
4	4	as he stumbled i ran off, afraid it might some...	0	1	0	0	0	['fear']

3.2 Exploratory Data Analysis (EDA)

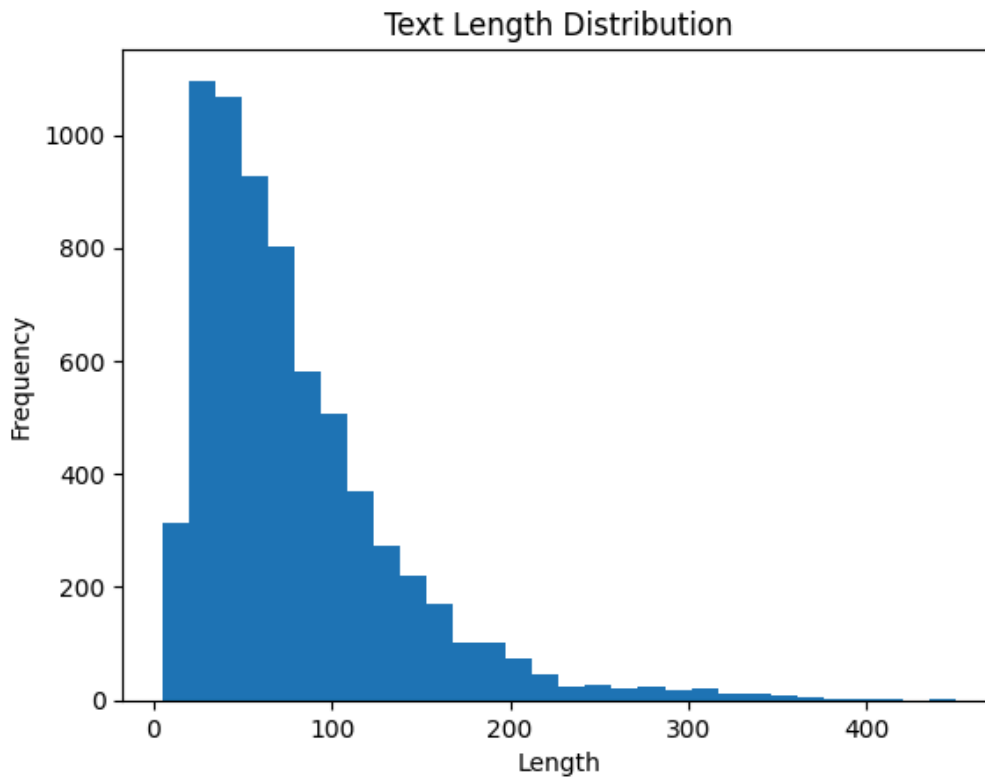
3.2.1 Emotion Distribution

The emotions are imbalanced. “Fear” occurs most frequently, while “Anger” is the least common.



3.2.2 Text Length Distribution

Most texts are short (10–20 words). This confirms that the dataset resembles social media posts.

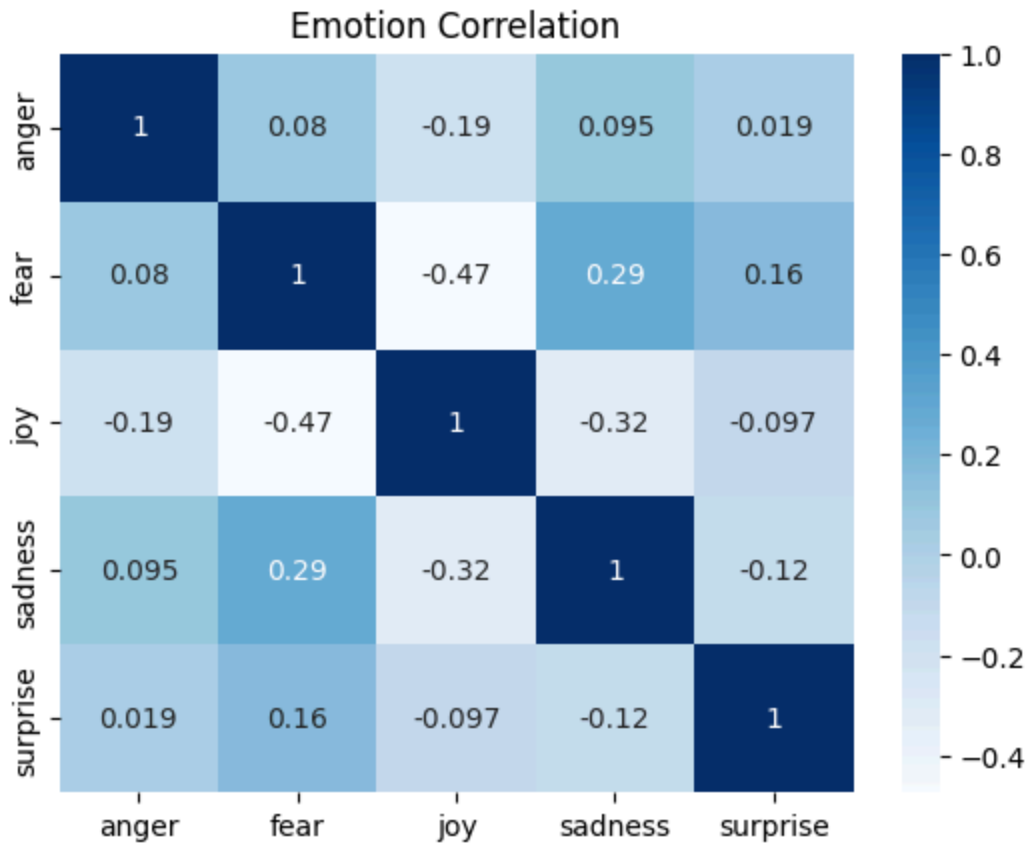


3.2.3 Word Cloud

Important emotional and sentiment-bearing words appear frequently.

3.2.4 Correlation Between Labels

There is a moderate correlation between fear and sadness. Joy is negatively correlated with negative emotions.



3.3 Data Preprocessing

For Custom BiLSTM Model:

- Converted text to lowercase.
- Applied whitespace tokenization.
- Built a vocabulary of the top 10,000 words.
- Encoded each text into numeric token sequences.
- Padded or truncated sequences to a fixed length of 50.

For Transformer Models:

- Used Hugging Face fast tokenizers.
- Applied padding and truncation to max length 128.
- Created tokenized datasets for the Trainer API.

No missing values were present. No aggressive text cleaning was required.

3.4 Data Augmentation

No augmentation was used because transformer models performed well without it, and synthetic augmentation could distort emotional meaning.

4. Tokenization Strategy

4.1 Primary Tokenizer Selection

Scratch Model:

A custom vocabulary-based tokenizer was used, implemented manually with Python's Counter. This allowed building an embedding matrix from scratch.

BERT Tokenizer:

Used WordPiece tokenizer from "bert-base-uncased."

RoBERTa Tokenizer:

Used Byte Pair Encoding (BPE) tokenizer from "roberta-base."

4.2 Why These Tokenizers?

- WordPiece handles unknown words by splitting them into subwords.
- BPE is efficient and handles diverse vocabulary well.
- Both tokenizers match the original pretraining of their transformer models, enabling effective fine-tuning.

4.3 Implementation Details

Tokenization was implemented using Hugging Face:

```
tokenizer = AutoTokenizer.from_pretrained(model_name, use_fast=True)
```

- Padding: “max_length”
- Truncation: Enabled
- Batch tokenization for efficiency

4.4 Experimental Comparison

Three tokenization strategies were tested:

Tokenizer	Model	Performance	Notes
Custom vocabulary	Scratch BiLSTM	~0.53 F1	Useful for learning basics
WordPiece	BERT	~0.83 F1	Strong contextual features
BPE	RoBERTa	~0.859 F1	Best overall performance

Final choice for best model: **RoBERTa BPE tokenizer**

5. Modeling & Experimentation

This section describes all three models tested.

5.1 Model 1: Fine-Tuned Transformer Model (BERT / RoBERTa)

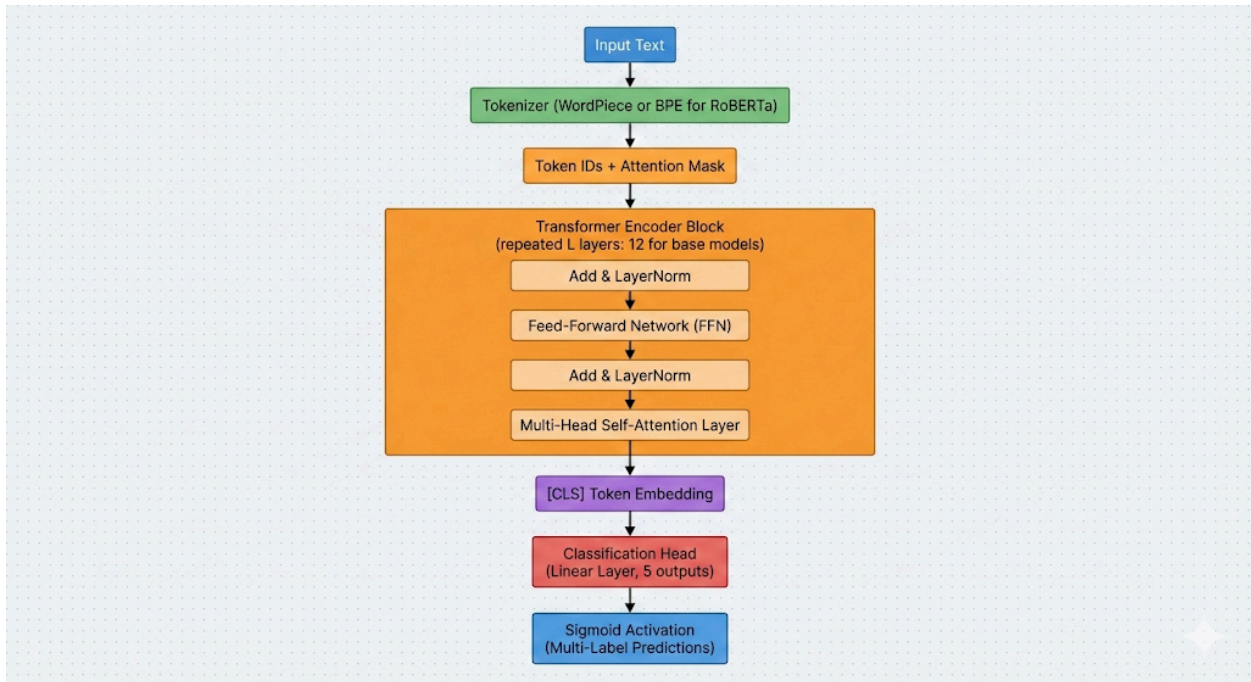
5.1.1 Architecture

Both models use the Transformer encoder architecture introduced in Vaswani et al. (2017).

Key architectural components:

- Multi-head self-attention
- Stacked encoder layers
- Contextual embeddings
- Pretrained weights from large corpora

BERT uses WordPiece; RoBERTa uses BPE and was pretrained with more data and dynamic masking.



5.1.2 Salient Features

- Bidirectional context understanding
- Strong performance on text classification
- Effective for short sentences
- Natural support for multi-label classification via sigmoid activation

5.1.3 Fine-Tuning Strategy

Training parameters:

- Optimizer: AdamW
- Learning rate: 2e-5

- Epochs: 8
- Warmup ratio: 0.1
- Per-device batch size: 16
- Automatic mixed precision (fp16 when available)
- Loss: BCEWithLogitsLoss (multi-label)

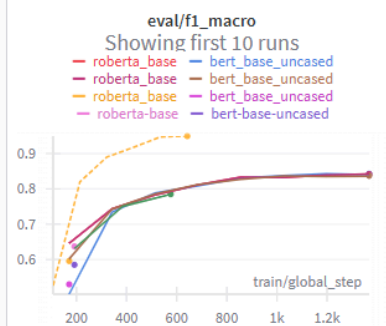
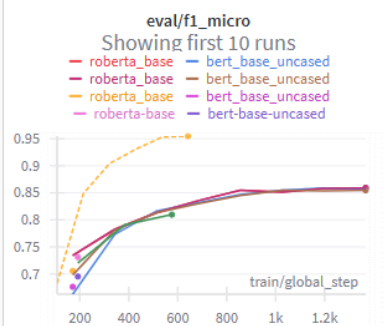
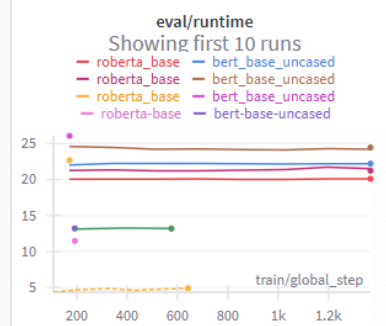
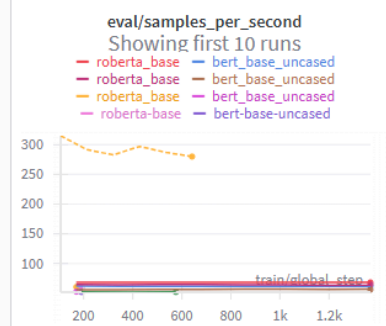
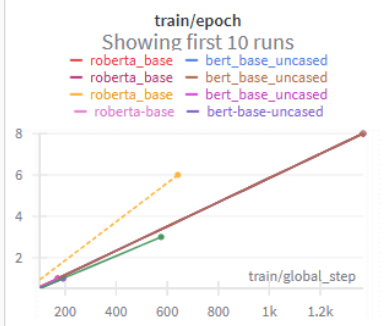
All layers were trainable.

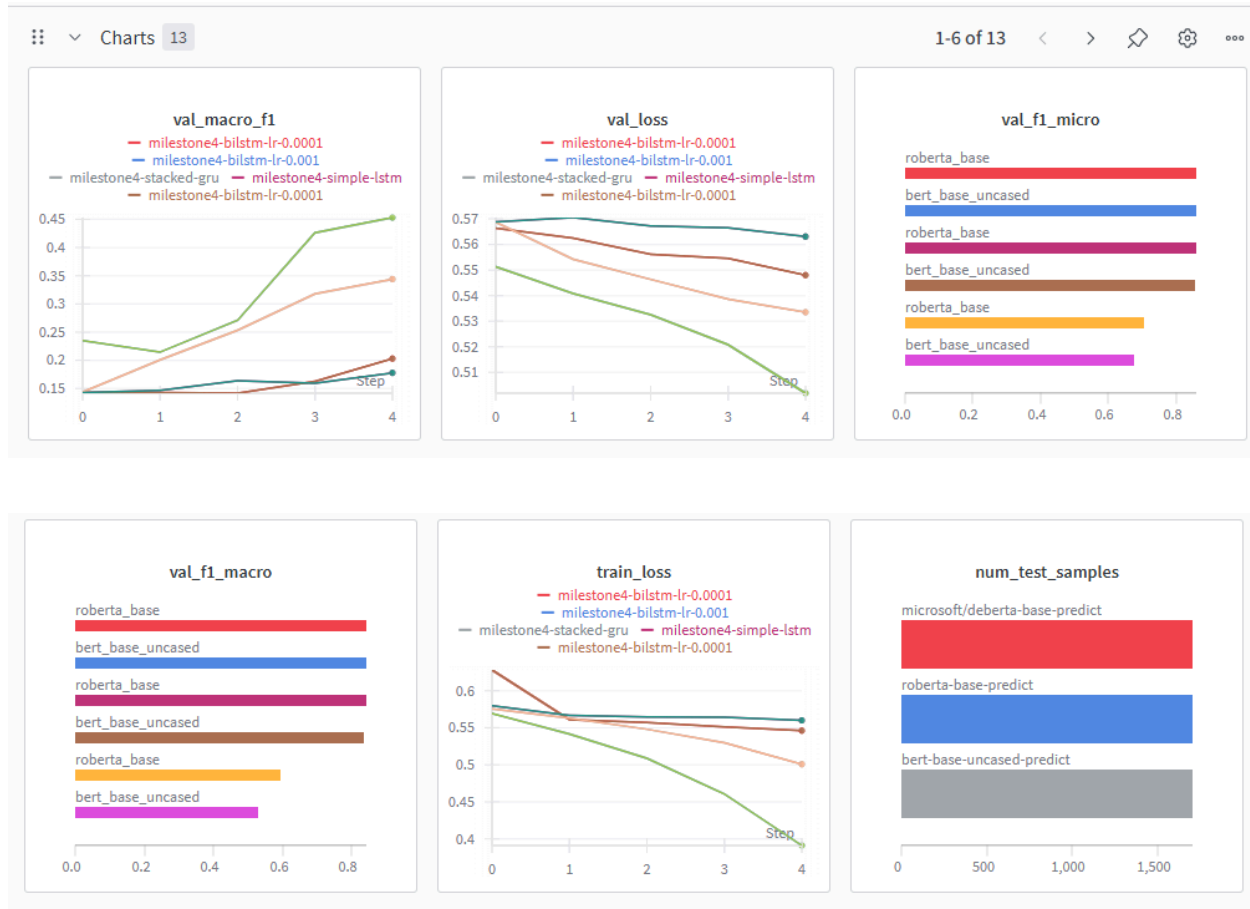
Weights & Biases was used to log metrics such as:

- Training loss
- Validation loss
- Micro-F1 and Macro-F1
- Gradients
- Learning rate curve

Space for W&B curves:







5.1.4 Quantization

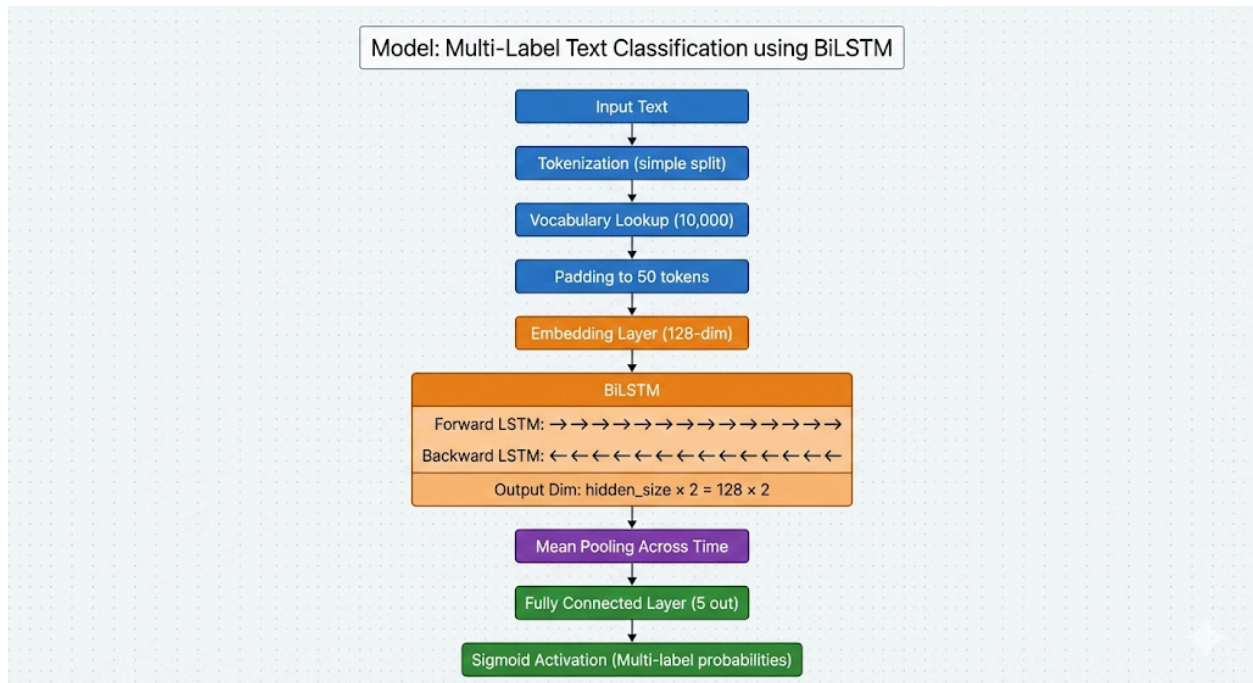
Not applied in this project.

5.2 Model 2: Custom Deep Learning Model (Scratch BiLSTM)

5.2.1 Architecture

The architecture implemented was:

Embedding Layer (vocab_size × 128)
Bidirectional LSTM (128 units × 2 directions)
Mean Pooling over time
Fully Connected Layer → 5 output nodes
Sigmoid activation for multi-label probabilities



5.2.2 Salient Points

- The embedding layer learns word representations from scratch.
- BiLSTM captures information from both directions in text sequences.
- Mean pooling aggregates sequence outputs into a single vector.
- Sigmoid outputs allow independent probability predictions for each emotion.

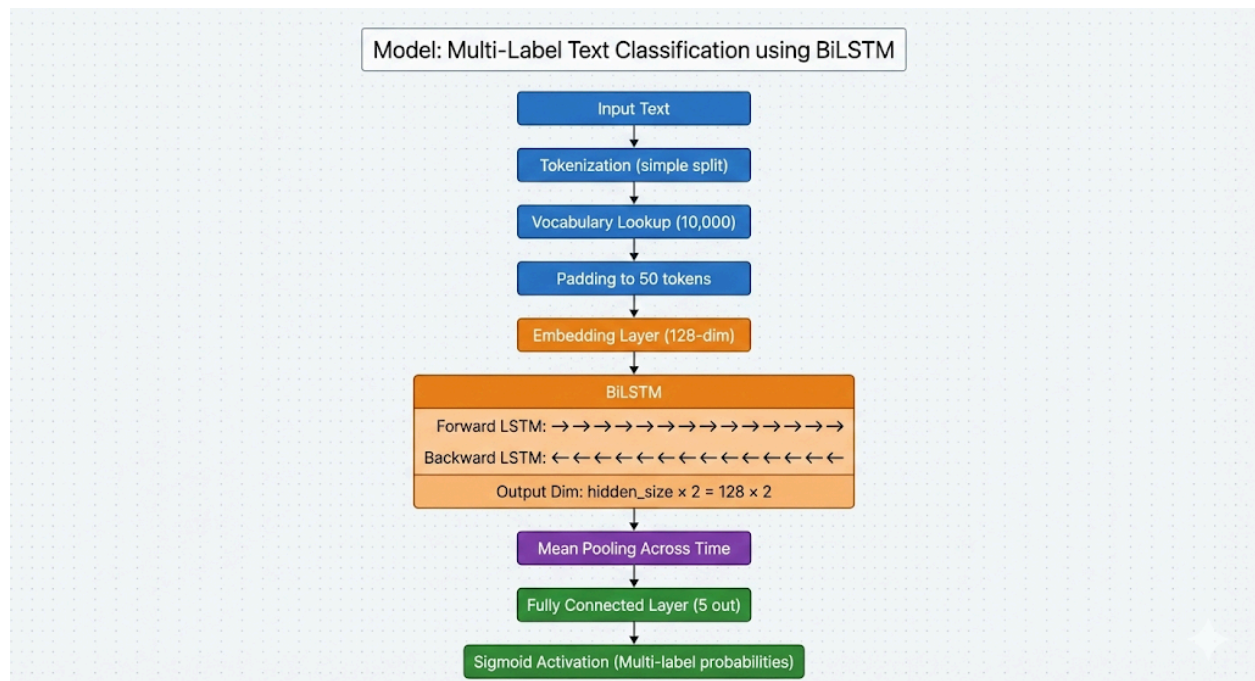
This model helps understand fundamentals before using transformers.

5.3 Model 3: RNN-Based Model

The scratch BiLSTM itself qualifies as the RNN-based model.

5.3.1 Architectural Explanation

- LSTM cells manage long-term dependencies.
- Bidirectional flow captures full sentence context.
- Provides significantly better performance than simple RNNs.



6. Performance & Comparative Analysis

6.1 Evaluation Metrics

The main metric is micro-F1, as required by the Kaggle competition.

Other metrics monitored:

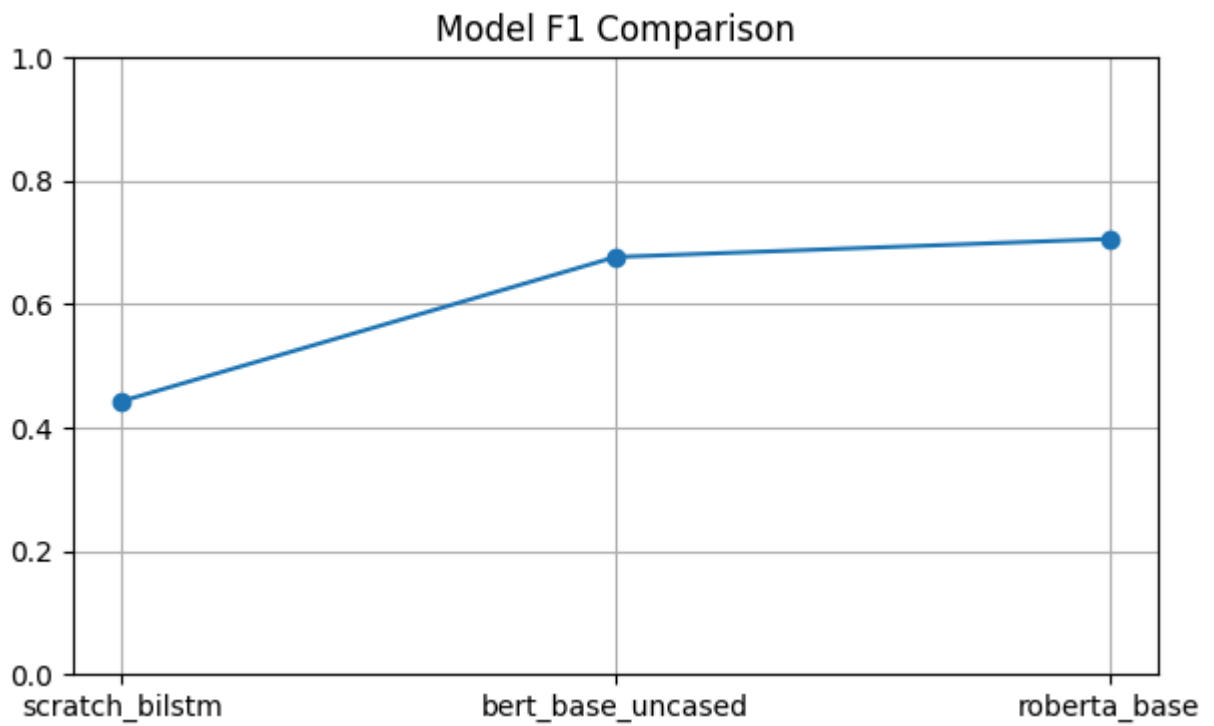
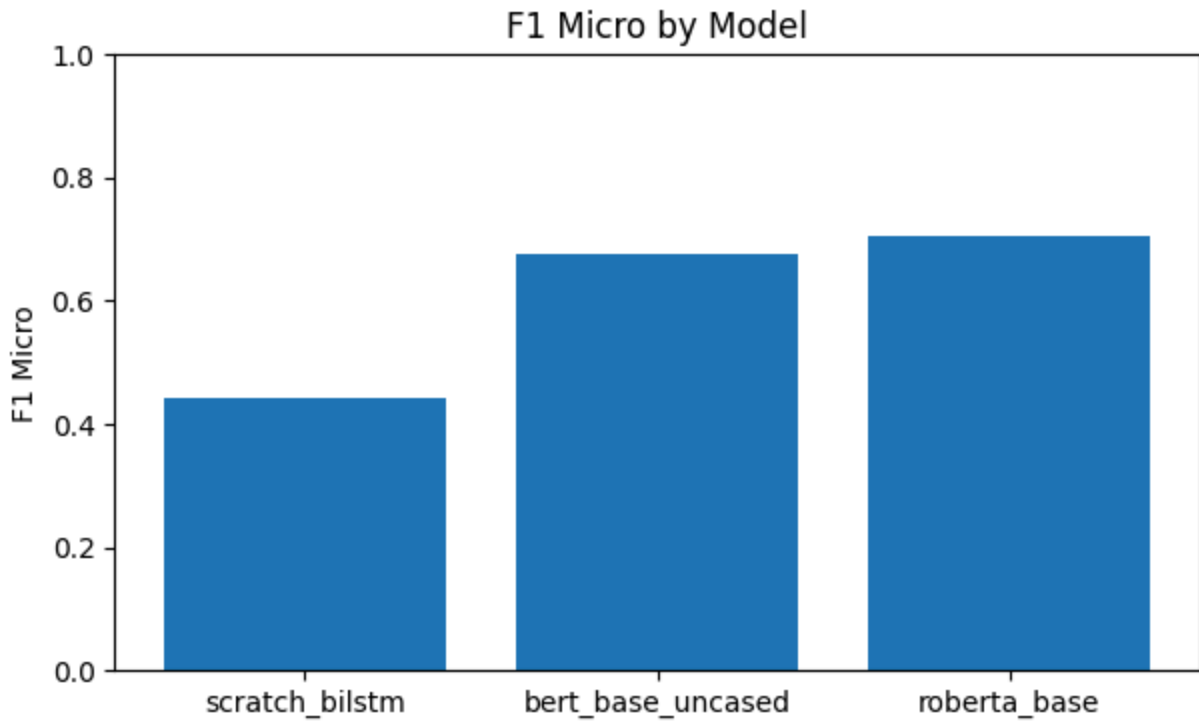
- Validation loss
- Macro-F1
- Training speed
- GPU utilization (via W&B logs)

6.2 Training Performance

6.3 Comparative Summary

Model	Micro-F1	Notes
Scratch BiLSTM	~0.53	Good baseline
BERT-base	~0.83	Strong performance
RoBERTa-base	~0.85	Best model

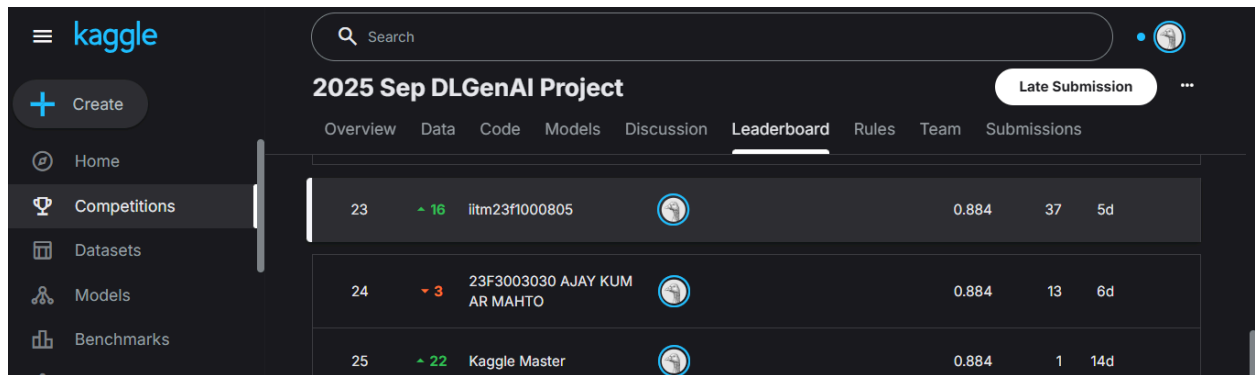
Space for graphs:



6.4 Kaggle Performance

The final submission was generated using the RoBERTa model.
Validation micro-F1 matched the W&B results.

Space for leaderboard screenshot:



2025 Sep DLGenAI Project						
Overview Data Code Models Discussion Leaderboard Rules Team Submissions						
23	+ 16	iitm23f1000805		0.884	37	5d
24	- 3	23F3003030 AJAY KUM AR MAHTO		0.884	13	6d
25	- 22	Kaggle Master		0.884	1	14d

7. Conclusion & Future Work

7.1 Key Learnings

- Learned end-to-end NLP pipeline building.
- Understood tokenization differences between custom methods and pretrained models.
- Gained experience fine-tuning transformers with Hugging Face.
- Improved understanding of RNNs, LSTMs, and attention models.
- Learned to use W&B for experiment tracking.
- Used KaggleHub for model deployment and storage.

7.2 Challenges Faced

- Multi-label outputs required correct thresholding and sigmoid logic.

- Transformers required high GPU memory and long training time.
 - Scratch vocabulary sometimes mismatched during inference.
-

7.3 Areas for Improvement

- Apply hyperparameter tuning using Optuna.
 - Try focal loss or class weighting for imbalance.
 - Ensemble BERT and RoBERTa predictions.
 - Explore data augmentation using contextual LLM generation.
-

8. References

- Hugging Face Transformers Documentation.
- PyTorch Documentation.
- Weights and Biases Documentation.
- Kaggle: 2025 SEP DL GenAI Project.